

Laboratorijska vježba 6 – DevOps (Terraform)

Cilj vježbe:

Upoznavanje studenata sa Terraform-om i osnovnim naredbama i konfiguracijama s ciljem automatizacije AWS procesa kreiranja.

Vježba je bazirana na:

https://gitlab.com/kibrovic/terraform-task/-/tree/arm?ref_type=heads

Prije izrade vježbe potrebno je:

1. Instalirati [aws-cli](#)
2. Instalirati [Terraform](#)
3. Konfigurirati AWS CLI – Konfiguracijski file se nalazi u AWS Details unutar Learner Lab
4. Kreirati [SSH ključ](#) za pristupanje instanci.
5. Pregledati [standardnu modularnu strukturu](#) koja će se koristiti.

Zadaci:

1. Kreirajte Terraform workspace: *terraform workspace new arm*
2. Kreirati Terraform konfiguraciju u sklopu koje će se:
 - a. Kreirati *arm_vpc*:
 - i. Koristiti 192.168.1.0/24
 - ii. Omogućiti DNS hostnames
 - iii. Podijeliti VPC u dvije jednake pod mreže, *arm_subnet_private* i *arm_subnet_public*
 - iv. Dozvoliti sav ulazni i izlazni HTTP i HTTPS saobraćaj
 - v. Dozvoliti SSH konekciju s vlastitog uređaja
 - b. Kreirati ECS kluster gdje će se deploy-ati aplikacije:
 - i. Kluster će koristiti t2.micro EC2 instance
 - ii. Autoskaliranje će biti konfigurisano na 1 početnu instancu s maksimalno 2 moguće instance
 - iii. Skaliranje se odnosi na javnu pod mrežu
 - iv. EC2 instanca baze se nalazi u privatnoj pod mreži
 - v. EC2 instancama se pristupa preko SSH protokola
 - vi. ECS kluster ima zadatke definisane za frontend i bazne servise
 - vii. Frontend servis se treba nalaziti u javnoj pod mreži
 - viii. Bazni servis se nalazi u privatnoj pod mreži
3. Kreirati plan egzekucije *terraform plan -out terraform.tfplan*
4. Izvršiti plan *terraform apply "terraform.tfplan"*
5. Uništavanje se izvršava primjenom naredbe: *terraform destroy*

Napomena: Poželjno je korišćenje [terraform varijabli](#) i [AWS terraform provider-a](#)

Detalji izrade:

1. Kreiranje main.tf konfiguracije za inicijalizaciju u sklopu koje će se definisati terraform required_providers i region. Nakon definisanja uraditi *terraform init*

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.40"
    }
  }

  required_version = ">= 1.2.0"
}

provider "aws" {
  region = var.aws_region
  default_tags {
    tags = {
      Project = "AdministracijaRacunarskihMreza"
    }
  }
}

locals {
  account_id = data.aws_caller_identity.current.account_id
}
```

- Definirati sljedeće varijable (variables.tf)

```
variable "aws_region" {
  type        = string
  description = "AWS Region"
}

variable "vpc_cidr" {
  type        = string
  description = "VPC CIDR Block"
}

variable "private_subnet" {
  type        = string
  description = "Private subnet"
}

variable "public_subnet" {
  type        = string
  description = "Public subnet"
}

variable "user_source_ip" {
  type        = string
  description = "User's source IP used for security group whitelist"
}
```

```
variable "ssh_pubkey" {
  type      = string
  description = "SSH public key used to access EC2 instances"
}
```

- Varijable se čuvaju sakrivene (koristiti .gitignore) unutar terraform.tfvars file-a:

```
aws_region = "us-east-1"
vpc_cidr = "192.168.1.0/26"
private_subnet = "192.168.1.0/27"
public_subnet = "192.168.1.32/27"
user_source_ip = "1.1.1.1/32"
ssh_pubkey = "ssh-ed25519 ED_25518_PUBKEY mail.com"
```

2. Kreirati mrežnu konfiguraciju (network.tf):

- Kreirati resurs arm_vpc koji će koristiti 192.168.1.0/26 CIDR i imati enable_dns_hostnames postavljen na true.
- Kreirati resurse arm_subnet_private i arm_subnet_public (koji će podijeliti postojeći CIDR na dva dijela)
- Kreirati resurs arm_igw za internet gateway.
- Kreirati resurs arm_public_rt koji rutira sav saobraćaj na arm_igw
- Spojiti arm_public_rt sa arm_subnet_public
- Kreirati resurs arm_eip koji kreira elastičnu IP adresu
- Kreirati resurs arm_nat_gateway koji kreira NAT gateway sa elastičnom IP adresom arm_eip koji se nalazi u javnoj podmreži
- Kreirati arm_private_rt koji će rutirati sav saobraćaj na NAT gateway.
- Povezati arm_private_rt sa arm_private_subnet
- Kreirati resurs arm_security_group:
 - Povezati s arm_vpc
 - Dozvoliti SSH s vlastite IP adrese
 - Dozvoliti HTTP i HTTPS sa svih destinacija
 - Dozvoliti sav ulazni i izlazni saobraćaj u sigurnosnim grupama

```
resource "aws_vpc" "arm_vpc" {
  cidr_block      = var.vpc_cidr
  enable_dns_hostnames = true
}
```

```
resource "aws_subnet" "arm_subnet_private" {
  vpc_id      = aws_vpc.arm_vpc.id
  cidr_block = var.private_subnet
}
```

```

    tags = {
        "Name" = "Private"
    }
}

resource "aws_subnet" "arm_subnet_public" {
    vpc_id            = aws_vpc.arm_vpc.id
    cidr_block        = var.public_subnet
    map_public_ip_on_launch = true
    tags = {
        "Name" = "Public"
    }
}

resource "aws_internet_gateway" "arm_igw" {
    vpc_id = aws_vpc.arm_vpc.id
}

resource "aws_route_table" "arm_public_rt" {
    vpc_id = aws_vpc.arm_vpc.id

    route {
        cidr_block = "0.0.0.0/0"
        gateway_id = aws_internet_gateway.arm_igw.id
    }
}

resource "aws_route_table_association" "arm_rt_public_subnet" {
    subnet_id      = aws_subnet.arm_subnet_public.id
    route_table_id = aws_route_table.arm_public_rt.id
}

resource "aws_eip" "arm_eip" {
    vpc = true
}

resource "aws_nat_gateway" "arm_nat_gateway" {
    allocation_id = aws_eip.arm_eip.id
    subnet_id     = aws_subnet.arm_subnet_public.id

    depends_on = [
        aws_internet_gateway.arm_igw
    ]
}

resource "aws_route_table" "arm_private_rt" {
    vpc_id = aws_vpc.arm_vpc.id

    route {
        cidr_block      = "0.0.0.0/0"
        nat_gateway_id = aws_nat_gateway.arm_nat_gateway.id
    }
}

resource "aws_route_table_association" "arm_rt_private_subnet" {
    subnet_id      = aws_subnet.arm_subnet_private.id
    route_table_id = aws_route_table.arm_private_rt.id
}

```

```

resource "aws_security_group" "arm_security_group" {
  name          = "arm_security_group"
  description   = "Security group for ARM EC2 instance"
  vpc_id       = aws_vpc.arm_vpc.id

  ingress {
    from_port    = 22
    to_port      = 22
    protocol     = 6 # TCP
    cidr_blocks  = [var.user_source_ip]
    description  = "Allow SSH ingress"
  }

  ingress {
    from_port    = 80
    to_port      = 80
    protocol     = 6
    cidr_blocks  = ["0.0.0.0/0"]
    description  = "Allow HTTP ingress"
  }

  ingress {
    from_port    = 443
    to_port      = 443
    protocol     = 6
    cidr_blocks  = ["0.0.0.0/0"]
    description  = "Allow HTTPS ingress"
  }

  ingress {
    from_port    = 0
    to_port      = 0
    protocol     = -1
    self         = true
    description  = "Allow traffic in security group"
  }

  egress {
    from_port    = 80
    to_port      = 80
    protocol     = 6
    cidr_blocks  = ["0.0.0.0/0"]
    description  = "Allow HTTP egress"
  }

  egress {
    from_port    = 443
    to_port      = 443
    protocol     = 6
    cidr_blocks  = ["0.0.0.0/0"]
    description  = "Allow HTTPS egress"
  }

  egress {
    from_port    = 0
    to_port      = 0
    protocol     = -1
  }

```

```

        self          = true
        description = "Allow traffic in security group"
    }
}

```

3. Kreirati IAM konfiguraciju (iam.tf) – Postoji samo jedna rola: LabRole-a

```

data "aws_caller_identity" "current" {}

data "aws_iam_role" "lab_role" {
    name = "LabRole"
}

data "aws_iam_instance_profile" "lab_instance_profile" {
    name = "LabInstanceProfile"
}

```

4. Konfiguracija ključeva (keys.tf):

- Kreirati resurs arm_ec2_access_key koji će kreirati EC2 par ključeva
- Kreirati ebs_encryption_key koji kreira kms_key. Isti služi za enkripciju EBS volumena EC2 instanci
 - Sve KMS akcije su dozvoljene trenutnom korisniku i AWSServiceRoleForAutoScaling (**Napomena: Ova rola ne postoji automatski u studentskom pristupu. Kako bi se kreirala ista, potrebno je kreirati testnu AutoScaleGroup**)
 - Koristiti aws_iam_policy_document resurs za deifnisanje politike ključeva

```

resource "aws_key_pair" "arm_ec2_access_key" {
    key_name     = "arm-ec2-access-key"
    public_key = var.ssh_pubkey
}

data "aws_iam_policy_document" "ebs_encryption_kms_key_policy" {
    statement {
        sid = "Allow all to root and terraform user"
        principals {
            type = "AWS"
            identifiers = [
                "${data.aws_caller_identity.current.arn}"
            ]
        }
        actions   = ["kms:*"]
        resources = ["*"]
    }
    statement {
        sid      = "Allow service-linked role use of the customer managed key"
        effect   = "Allow"
        principals {
            type = "AWS"

```

```

        identifiers = [
            "${data.aws_caller_identity.current.arn}",
            "arn:aws:iam::${local.account_id}:role/aws-service-role/autoscaling
.amazonaws.com/AWSServiceRoleForAutoScaling"
        ]
    }
    actions = [
        "kms:Encrypt",
        "kms:Decrypt",
        "kms:ReEncrypt*",
        "kms:GenerateDataKey*",
        "kms:DescribeKey"
    ]
    resources = ["*"]
}

statement {
    sid      = "Allow attachment of persistent resources"
    effect   = "Allow"
    principals {
        type = "AWS"
        identifiers = [
            "${data.aws_caller_identity.current.arn}",
            "arn:aws:iam::${local.account_id}:role/aws-service-role/autoscaling
.amazonaws.com/AWSServiceRoleForAutoScaling"
        ]
    }
    actions      = ["kms:CreateGrant"]
    resources    = ["*"]
    condition {
        test      = "Bool"
        variable   = "kms:GrantIsForAWSResource"
        values     = [true]
    }
}
}

resource "aws_kms_key" "ebs_encryption_key" {
    description = "Encryption Key for EBS volume"
    policy      = data.aws_iam_policy_document.ebs_encryption_kms_key_policy.json
}

```

5. Konfiguracija EC2 (ec2.tf):

- Definirati izvor podataka `ecs_optimized_amazon_linux_ami` koji sadrži informacije o AWS AMI instanci (koristit će se najnovija s arhitekturom `x86_64` i root device tipom `EBS`)
- Kreirati resurs `arm_launch_template` za potrebe Autoscaling grupe:
 - Koristiti `ecs_optimized_amazon_linux_ami`
 - Koristiti `t2.micro` tip instance

- Koristiti arm_ec2_access_key ključ za SSH pristup
- Instanca se treba deployati unutar VPC-a s arm_security_group
- ECS agent započinje s default cluster konfiguracijom. Potrebno je promijeniti na arm_ecs_cluster. Korisnički podaci se mogu pružiti kroz init skriptu
 - Kreirati templates folder s file-ovm user_data.tpl

```
#!/bin/bash
```

```
echo "Configuring ECS agent"
cat <<'EOF' | sudo tee /etc/ecs/ecs.config
ECS_CLUSTER=${cluster_name}
ECS_CONTAINER_INSTANCE_PROPAGATE_TAGS_FROM=ec2_instance
EOF
```

- Enkriptovati EBS korištenjem ebs_encryption_key
- Dodati lab_instance_profile
- Dodati tag Name=PublicServer
- Kreirati resurs arm_autoscaling_group koji definiše broj dostupnih EC2 instanci:
 - Minimum je 1, maksimum 2
 - Instance su unutar arm_subnet_public
 - Najnoviji arm_launch_template se koristi
 - Dodati tag Name=PublicServer
- Kreirati arm_server_private koji kreira EC2 instancu baze:
 - Koristi se ecs_optimized_amazon_linux_ami slika
 - Koristi se t2.micro tip
 - Instanca će biti deployana unutar arm_security_group
 - ECS agent započinje s default cluster konfiguracijom. Potrebno je promijeniti na arm_ecs_cluster. Korisnički podaci se mogu pružiti kroz init skriptu.
 - Enkriptovati EBS korištenjem ebs_encryption_key
 - Dodati lab_instance_profile
 - Dodati tag Name=PrivateServer

```
data "aws_ami" "ecs_optimized_amazon_linux_ami" {
  most_recent = true

  filter {
    name = "name"
```

```

    values = ["amzn2-ami-ecs-hvm-*"]
}

filter {
    name    = "root-device-type"
    values = ["ebs"]
}

filter {
    name    = "architecture"
    values = ["x86_64"]
}

owners = ["591542846629"]
}

resource "aws_launch_template" "arm_launch_template" {
    name_prefix      = "arm_launch_template"
    image_id         = data.aws_ami.ecs_optimized_amazon_linux_ami.id
    instance_type    = "t2.micro"
    key_name         = aws_key_pair.arm_ec2_access_key.key_name
    vpc_security_group_ids = [aws_security_group.arm_security_group.id]
    user_data        = base64encode(templatefile("./templates/user_data.tpl", {
        cluster_name = aws_ecs_cluster.arm_ecs_cluster.name
    }))
    update_default_version = true

    block_device_mappings {
        device_name = "/dev/xvda"
        ebs {
            delete_on_termination = true
            encrypted             = true
            kms_key_id            = aws_kms_key.ebs_encryption_key.arn
            volume_size           = 30
        }
    }

    iam_instance_profile {
        name = data.aws_iam_instance_profile.lab_instance_profile.name
    }

    depends_on = [
        aws_kms_key.ebs_encryption_key
    ]

    lifecycle {
        create_before_destroy = true
    }

    tags = {
        "Name" = "PublicServer"
    }
}

resource "aws_autoscaling_group" "arm_autoscaling_group" {
    name_prefix      = "arm_autoscaling_group"
    max_size         = 2
    min_size         = 1

```

```

vpc_zone_identifier      = [aws_subnet.arm_subnet_public.id]
wait_for_capacity_timeout = "2m"

launch_template {
  id      = aws_launch_template.arm_launch_template.id
  version = "$Latest"
}

tag {
  key     = "Name"
  value   = "PublicServer"
  propagate_at_launch = true
}

depends_on = [
  aws_launch_template.arm_launch_template
]

resource "aws_instance" "arm_server_private" {
  ami                  = data.aws_ami.ecs_optimized_amazon_linux_ami.id
  iam_instance_profile = data.aws_iam_instance_profile.lab_instance_profile.name
  instance_type        = "t2.micro"
  vpc_security_group_ids = [resource.aws_security_group.arm_security_group.id]
  subnet_id            = aws_subnet.arm_subnet_private.id
  user_data_base64     = base64encode(templatefile("./templates/user_data.tpl",
{
  cluster_name = aws_ecs_cluster.arm_ecs_cluster.name
})))

  root_block_device {
    encrypted = true
    kms_key_id = resource.aws_kms_key.ebs_encryption_key.arn
  }

  tags = {
    "Name" = "PrivateServer"
  }
}

```

6. Kreirati ECS konfiguraciju (ecs.tf):

- Kreirati resurs `arm_ecs_cluster`
- Kreirati resurse `frontend_task_definition` i `database_task_definition` unutar kojih će biti definisane konfiguracije kontejnera
- Kreirati resurs `frontend_service` i `database_service` koji će voditi računa o task-ovima.
- **Napomena: Taskovi su ovdje kreirani kao primjer - detaljno će biti predstavljeni isti na sljedećoj vježbi!**

```

resource "aws_ecs_cluster" "arm_ecs_cluster" {
  name = "arm_ecs_cluster"

  setting {
    name = "containerInsights"
    value = "disabled"
  }
}

resource "aws_ecs_task_definition" "frontend_task" {
  container_definitions = jsonencode(
    [
      {
        command      = []
        cpu           = 256
        entryPoint    = []
        environment   = [
          {
            name  = "MYSQL_DB"
            value = aws_instance.arm_server_private.private_dns
          },
        ]
        essential    = true
        image         = "docker io image"
        memory        = 512
        mountPoints   = []
        name          = "frontend-task"
        portMappings = [
          {
            containerPort = 8080
            hostPort      = 80
            protocol       = "tcp"
          },
        ]
        volumesFrom = []
      },
    ]
  )
  execution_role_arn = aws_iam_role.ecs_task_execution_role.arn
  family             = "frontend-task"
  requires_compatibilities = [
    "EC2",
  ]
  task_role_arn = aws_iam_role.ecs_task_execution_role.arn

  placement_constraints {
    expression = "attribute:ecs.subnet-id in [{aws_subnet.arm_subnet_public.id}]"
    type       = "memberOf"
  }
}

resource "aws_ecs_service" "frontend_service" {
  cluster                = aws_ecs_cluster.arm_ecs_cluster.id
  deployment_maximum_percent = 100
  deployment_minimum_healthy_percent = 0
  desired_count           = 1
  name                    = "frontend-service"

```

```

        task_definition
aws_ecs_task_definition.frontend_task.arn
}

resource "aws_ecs_task_definition" "database_task" {
  container_definitions = jsonencode(
    [
      {
        command      = []
        cpu           = 256
        entryPoint    = []
        environment   = [
          {
            name  = "MYSQL_DATABASE"
            value = "DBWT19"
          },
          {
            name  = "MYSQL_ROOT_PASSWORD"
            value = "root"
          },
        ],
        essential = true
        healthCheck = {
          command = [
            "mysqladmin ping -h localhost",
          ]
          interval = 30
          retries  = 10
          timeout  = 20
        }
        image      = "mysql"
        memory     = 512
        mountPoints = []
        name       = "database"
        portMappings = [
          {
            containerPort = 3306
            hostPort       = 3306
            protocol       = "tcp"
          },
        ],
        volumesFrom = []
      },
    ]
  )
  execution_role_arn = aws_iam_role.ecs_task_execution_role.arn
  family             = "database-task"
  requires_compatibilities = [
    "EC2",
  ]
  task_role_arn = aws_iam_role.ecs_task_execution_role.arn

  placement_constraints {
    expression = "attribute:ecs.subnet-id in [{aws_subnet.arn_subnet_private.id}]"
    type       = "memberOf"
  }
}

```

```
resource "aws_ecs_service" "database_service" {
  cluster                = aws_ecs_cluster.arm_ecs_cluster.id
  deployment_maximum_percent = 100
  deployment_minimum_healthy_percent = 0
  desired_count          = 1
  name                    = "database-service"
  task_definition          = aws_ecs_task_definition.database_task.arn
}
```